WEEK 3 · LESSON 18 OF 24 · HANDOUT

Week 3 Project: An Image Classifier from Folders

Study time: **about 120 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=WIhI1W6NoZ0 — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython: How to train an image classifier using PyTorch — https://www.youtube.com/watch?v=eDRck258bnw

🎓 **Go deeper:** freeCodeCamp: PyTorch for Deep Learning & Machine Learning — full course — https://www.youtube.com/watch?v=V_xro1bcAuA

Your goal in this lesson: Build a complete image-classification project the way it is done in industry: images in folders, train/validation/test splits, augmentation, a pretrained network, early stopping and an honest evaluation.

The brief

A farmers' cooperative wants a phone app that recognises plant conditions from photos. Before collecting thousands of photos, it asks you to build the full **pipeline** on a small sample, so that real photos can simply be dropped in later.

Real image projects store pictures in **one folder per class**:

```
photos/
    healthy/      img001.jpg, img002.jpg, ...
    disease_a/    ...
    disease_b/    ...
```

To keep this project runnable for everyone, we first **write our shapes to folders as PNG files**. Later, point the same code at your own photos (for example from your phone, Google Drive, or a public dataset such as PlantVillage on Kaggle).

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

def make_shapes(n, kinds=("circle", "square", "triangle"), size=32, seed=0):
    # n noisy grey images of random shapes at random places and sizes
    rng = np.random.default_rng(seed)
    yy, xx = np.mgrid[0:size, 0:size]
    images = np.zeros((n, size, size), dtype=np.float32)
    labels = rng.integers(0, len(kinds), n)
    for i, lab in enumerate(labels):
        r = rng.uniform(5, 10)
        cx, cy = rng.uniform(r + 1, size - r - 1, 2)
        dx, dy = xx - cx, yy - cy
        kind = kinds[lab]
        if kind == "circle":
            mask = dx ** 2 + dy ** 2 <= r ** 2
        elif kind == "square":
            mask = (abs(dx) <= r * 0.85) & (abs(dy) <= r * 0.85)
        elif kind == "triangle":
            mask = (dy >= -r) & (dy <= r) & (abs(dx) <= (dy + r) / 2)
        elif kind == "ring":
            mask = (dx ** 2 + dy ** 2 <= r ** 2) & (dx ** 2 + dy ** 2 >= (0.55 * r) ** 2)
        elif kind == "diamond":
            mask = abs(dx) + abs(dy) <= r
        else:  # cross
            mask = ((abs(dx) <= 0.3 * r) & (abs(dy) <= r)) | ((abs(dy) <= 0.3 * r) & (abs(dx)
<= r))
        images[i][mask] = rng.uniform(0.6, 1.0)
        images[i] += rng.normal(0, 0.15, (size, size))
    images = torch.tensor(np.clip(images, 0, 1)).unsqueeze(1)   # (n, 1 channel, 32, 32)
    return images, torch.tensor(labels)

X_all, y_all = make_shapes(3000)
X_train, y_train = X_all[:2400], y_all[:2400]
X_test, y_test = X_all[2400:], y_all[2400:]
names = ["circle", "square", "triangle"]

fig, axes = plt.subplots(1, 8, figsize=(12, 2))
for ax, img, lab in zip(axes, X_train, y_train):
    ax.imshow(img[0], cmap="gray")
    ax.set_title(names[lab])
    ax.axis("off")
plt.show()
print(X_train.shape, torch.bincount(y_train))
```

Part 1 — Images into folders

```
from pathlib import Path
from PIL import Image

root = Path("shape_photos")
X_demo, y_demo = make_shapes(600, seed=5)
for i, (img, lab) in enumerate(zip(X_demo, y_demo)):
    folder = root / names[lab]
    folder.mkdir(parents=True, exist_ok=True)
    Image.fromarray((img[0].numpy() * 255).astype("uint8")).convert("RGB").save(folder / f"
{i:04d}.png")
print(sorted(p.name for p in root.iterdir()), sum(1 for _ in root.rglob("*.png")), "images")
```

Part 2 — Load with ImageFolder and split

```
from torchvision import datasets, models
from torchvision.transforms import v2
from torch.utils.data import DataLoader, Subset

weights = models.ResNet18_Weights.DEFAULT
base_tf = weights.transforms()                    # resize + normalise for ResNet

train_tf = v2.Compose([v2.RandomHorizontalFlip(), v2.RandomRotation(15), base_tf])

full_train = datasets.ImageFolder(root, transform=train_tf)
full_eval = datasets.ImageFolder(root, transform=base_tf)
print(full_train.classes, len(full_train))

g = torch.Generator().manual_seed(0)
order = torch.randperm(len(full_train), generator=g).tolist()
n_val = n_test = len(order) // 5
val_idx, test_idx, train_idx = order[:n_val], order[n_val:n_val + n_test], order[n_val +
n_test:]

train_loader = DataLoader(Subset(full_train, train_idx), batch_size=32, shuffle=True)
val_loader = DataLoader(Subset(full_eval, val_idx), batch_size=64)
test_loader = DataLoader(Subset(full_eval, test_idx), batch_size=64)
print(len(train_idx), "train /", len(val_idx), "val /", len(test_idx), "test")
```

`ImageFolder` reads the class names from the folder names. Two copies of the dataset let the training split be augmented while validation and test stay untouched.

Part 3 — A pretrained ResNet with a new head

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model = models.resnet18(weights=weights)
for p in model.parameters():
    p.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, len(full_train.classes))
model = model.to(device)
opt = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
print("device:", device)
```

Part 4 — Train with early stopping

```
import copy

def evaluate(loader):
    model.eval()
    correct = total = 0
    with torch.no_grad():
        for xb, yb in loader:
            pred = model(xb.to(device)).argmax(dim=1).cpu()
            correct += (pred == yb).sum().item()
            total += len(yb)
    return correct / total

best_acc, best_state, waited = 0.0, None, 0
for epoch in range(3):                        # use 10–20 epochs on a GPU with real photos
    model.train()
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        opt.zero_grad()
        F.cross_entropy(model(xb), yb).backward()
        opt.step()
    val_acc = evaluate(val_loader)
    print(f"epoch {epoch}: validation accuracy {val_acc:.3f}")
    if val_acc > best_acc:
        best_acc, best_state, waited = val_acc, copy.deepcopy(model.state_dict()), 0
    else:
        waited += 1
        if waited >= 3:
            break
model.load_state_dict(best_state)
print("test accuracy (used once):", round(evaluate(test_loader), 3))
```

On a Colab GPU each epoch takes seconds. A pretrained ResNet usually reaches high accuracy on such clear classes within a few epochs.

Part 5 — Predict a single new photo

```
def predict_photo(path):
    img = Image.open(path).convert("RGB")
    x = base_tf(img).unsqueeze(0).to(device)
    model.eval()
    with torch.no_grad():
        p = F.softmax(model(x), dim=1)[0].cpu()
    best = int(p.argmax())
    return full_train.classes[best], round(float(p[best]), 3)

example = next(root.rglob("*.png"))
print(example.parent.name, "->", predict_photo(example))
```

Part 6 — Evaluate and report (two pages)

1. **Data:** classes, number of images per class, how the split was made.
2. **Model:** pretrained network, what was frozen, augmentation used.
3. **Results:** validation curve, test accuracy, confusion matrix, five mistakes with comments.
4. **Trust:** saliency maps for three images (Lesson 17); confidence on right vs wrong answers.

5. **Field plan:** how you would collect real photos in Rwanda (phones, places, seasons, farmers' consent), a field test set, and a rule for sending low-confidence photos to an agronomist.

Marking guide (self-check)

Area	Excellent work
Pipeline	Folders → ImageFolder → separate train / val / test
Model	Pretrained network adapted with a new head
Training	Augmentation on training only; early stopping with best weights
Evaluation	Test used once; confusion matrix and mistakes studied
Responsibility	Field data plan, consent and human review for low confidence

Remember this: A real image project is a pipeline — folders, transforms, a pretrained model, careful splits and honest evaluation — that works the same for shapes and for real photos.

Name: _____ Date: _____

Activity

Task 1

Complete Parts 1–6. Then replace the `shape_photos` folder with at least three classes of your own photos (30+ per class, taken with your phone) and run the same notebook. What changes, and what accuracy do you get?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. How does ImageFolder know the class names?
   - A. You must type them
   - B. From the image colours
   - C. From the names of the sub-folders
   - D. From a CSV file
2. Why create two copies of the dataset with different transforms?
   - A. To save memory
   - B. To double the data
   - C. So training images are augmented while validation and test stay untouched
   - D. ImageFolder requires two
3. What must new photos go through before prediction?
   - A. Augmentation
   - B. Nothing
   - C. Flipping
   - D. The same preprocessing (resize, normalise) used for training
4. Why is the test accuracy computed only once at the end?
   - A. It changes the weights
   - B. PyTorch forbids more
   - C. To keep it an honest estimate
   - D. It is slow
5. What should happen to low-confidence predictions in a farm app?
   - A. Delete the photo
   - B. Send them to a human expert such as an agronomist
   - C. Hide them
   - D. Always say 'healthy'
6. Which part of the pretrained ResNet trains in this project?
   - A. Only the new final layer (fc)
   - B. Nothing
   - C. Only the first convolution
   - D. Every layer

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.